Java OOP Laboratory Exercise: Campus Café Ordering System

ITS 202: Object Oriented Programming

Submitted by:

Anamae K. Acabal

John Ivan Cacho

Section: C

NOVEMBER 10, 2026

Concept Explanation

1. **if Statement**: We used if statements where an action should only happen when a condition is true, such as adding an extra charge only when a pastry is warmed or a meal is upsized. We also used them in the Order class to check whether an item slot is empty before processing it.

2. **if-else Statement**: We used if-else statements when the program had to choose between two outcomes. For example, we reject negative prices in setBasePrice() (the output shows “Error: base price cannot be negative.”), refuse new items once the order is full, and apply either the student discount or a voucher when computing the total.

3. **Classes and Objects**: Our program is made of separate classes such as MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. In CafeApp we created the customer, menu item, and order objects and showed how the parts of the ordering system work together.

4. **Attributes and Behaviors**: Attributes store information about each object, such as the size of a beverage or whether a pastry is warmed. Behaviors are the methods that define what an object can do, like calculating its price with computePrice() or adding items to an order with addItem().

5. **Object Interaction (Order)**: The Order class connects a customer with up to three menu items. When printReceipt() is called, it gets the details and prices of the ordered items, computes the subtotal and total, applies the discount or voucher, and updates the customer’s loyalty points.

6. **Java Naming Conventions**: We used PascalCase for class names (MenuItem, CafeApp) and camelCase for methods and variables (basePrice, computeTotal).

7. **Default Constructor**: A default constructor gives an object starting values when no information is provided, for example setting a menu item’s name to “Unnamed Item” and its base price to 0.0.

8. **Parameterized Constructor**: Parameterized constructors let us supply an object’s information when it is created. In the subclasses we also used super(...) to pass the item’s name, price, and code to the parent MenuItem class.

9. **Constructor Overloading**: The Customer class has several constructors so a customer can be created in different ways: with no information, with only a name, or with both a name and student status.

10. **public / private / protected**: Access modifiers control how variables and methods can be reached. Most instance variables are private, while members such as name and basePrice are protected so that subclasses can use them.

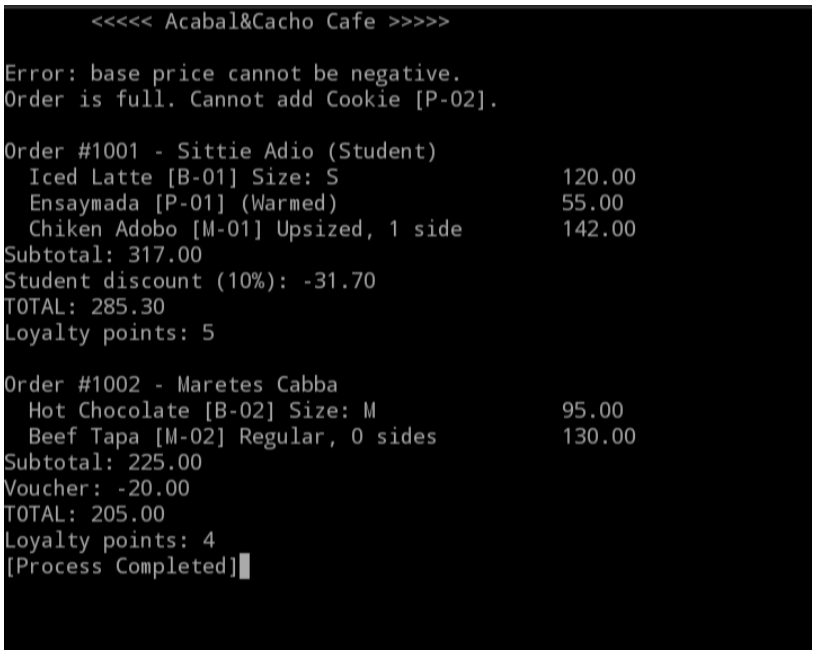
11. **Getters, Setters, and Data Hiding**: Getters and setters are used to read and change attributes instead of making variables public. Some setters include validation to prevent invalid values, such as negative prices and negative loyalty points.

12. **Method Overloading**: We created methods with the same name but different parameters, such as addPoints(int) and addPoints(double), computeTotal() and computeTotal(double), and printReceipt() and printReceipt(double). The overloaded versions handle receipts with and without a voucher amount.

13. **Method Overriding**: Beverage, Pastry, and Meal each provide their own computePrice() and getDescription() instead of using a single version from MenuItem, marked with the @Override annotation. This is why each item prints differently on the receipt (for example, “Size: S”, “(Warmed)”, and “Upsized, 1 side”).

14. **Abstraction**: MenuItem is an abstract class because it is only the parent of the actual menu items. Its abstract method computePrice() forces every subclass to define its own way of computing the price.

Screenshot / Output:



Output explanation: The program first shows the error message for an invalid negative base price, then the message that an order is full when a fourth item (Cookie [P-02]) is added. Order #1001 is a student order: the subtotal of 317.00 receives a 10% student discount of 31.70, giving a total of 285.30 and 5 loyalty points (one point for every 50 pesos). Order #1002 uses a 20.00 voucher on a subtotal of 225.00, giving a total of 205.00 and 4 loyalty points.

Challenges Encountered:

- **Comparing Strings**: When checking the drink size, == does not compare the contents of strings in Java. We used .equals() for each comparison and joined the conditions with ||.
- **Invalid Size**: If an invalid size is passed to the Beverage constructor and the setter rejects it, the size stays null, which can cause an error in computePrice(). We assigned a default size before checking the given input.
- **Abstract Method**: MenuItem must be declared abstract, and computePrice() must only be declared (no method body) so that each subclass is required to implement it.
- **Loyalty Points**: The addPoints() methods must add to the existing points using += instead of replacing them, reject negative values, and convert the amount spent into whole points (one point for every 50 pesos).
- **Computing Subtotal and Total**: We used variables to keep track of the subtotal and total, updating them during the calculation and returning the final values at the end of the methods.